

1) Explain **authentication and authorization**?

Authentication:-

Authentication is the process of verifying a user's identity to confirm that they are who they claim to be.

Authorization:-

Authorization is the process of determining what actions, resources, or permissions an authenticated user is allowed to access.

Feature	Authentication	Authorization
Meaning	Proves <i>who you are</i>	Decides <i>what you can access</i>
Question answered	“Are you really this user?”	“What is this user allowed to do?”
Purpose	Identity verification	Permission/Access control
When it happens	First step	After authentication
Based on	Password, OTP, biometrics, Google login, JWT	Roles, permissions, access rules
Example	Logging in with email + password	Admin can delete users; normal user cannot

For secure **Authentication and Authorization**

- JW Tokens
- Session-Based Authentication
- OAuth 2.0 (Google login, Facebook login, GitHub login)
- OpenID Connect

Why JWT is popular:

- Stateless
- Fast
- Works for web + mobile
- Good for distributed systems
- Easy to store roles/permissions inside

2) Explain Stateful and Stateless?

Stateful means the server stores user session or state information. The server remembers the user across multiple requests.(its stores is memory or database)

Stateless means the server does not store any user session or state information.

3) What is JWT (JSON Web Token)?

A **JWT (JSON Web Token)** is a compact, URL-safe, digitally signed token used to verify the identity of a user and to decide what they are allowed to do.

It travels with every request like a small encrypted ID card.

Compact: - They carry user information, but in a **very small and efficient** format so they can travel through the internet quickly.

URL-safe:- Some characters break URLs, like /, ?, +, =. JWT avoids those troublemakers.

Digitally signed:- A JWT is **signed using a secret key (HMAC)** or **private key (RSA/ECDSA)**.

A JWT has three parts, each base64 encoded and separated by dots . :-

header.payload.signature

➤ Header

A quick label telling what's inside.

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}  
  
alg: Algorithm used  
typ: Token type
```

➤ Payload

This is the claims section – like the details on an ID card.

```
{  
  "userid": 12,  
  "email": "ramani@example.com",  
  "role": "admin",  
  "exp": 1732132142  
}
```

exp is the expiry time

Payload is **NOT encrypted**. Anyone can read it. But they cannot **modify** it because signature will break.

➤ Signature

```
HMACSHA256(  
  base64(header) + "." + base64(payload),  
  secret_key  
)
```

If someone tries to edit the payload, signature becomes invalid.

4) How to Install the library

npm install jsonwebtoken

5) How to create token

Jwt.sign(payload, SECRET_KEY, {expiresIn: "1h"})

6) Explain access token and refresh token

Feature	Access Token	Refresh Token
Purpose	Authenticate user on every API request	Generate new access tokens when they expire
Lifetime	Short (5–15 minutes)	Long (7–30 days)
Security Level	Less sensitive	Highly sensitive
Where Used?	Sent in Authorization: Bearer <token> on every API call	Sent only to /refresh endpoint
Storage Location	Memory / localStorage / HttpOnly cookie	HttpOnly Secure cookie (recommended)
If Stolen?	Small damage (valid only a few minutes)	Dangerous (can create new access tokens)
Who Creates It?	Auth server during login or refresh	Auth server during login
When Sent to Server?	Every request needing authentication	Only when access token expires
Contains User Data?	Yes (userid, role, etc.)	Usually minimal OR opaque (ID only)
Visible to Client?	Yes	Usually hidden via HttpOnly cookie
Can It Be Revoked?	Hard (stateless)	Yes (server stores refresh tokens, can block/rotate)
Used For	Normal API access	Session continuation (silent re-login)
Expires Fast?	Yes	No
Should It Be Stored in localStorage?	Yes (acceptable)	No (never)

Simple Definition:

Access Token:

Your temporary visitor badge. Works inside the building. Expires quickly.

Refresh Token:

The permanent access form kept inside a locked cupboard.

Used only to issue new visitor badges.

- Access token keeps things fast & safe
- Refresh token keeps user logged in without asking password again
- Together they form **secure long-term authenticated sessions**

7) What Are Cookies?

Cookies are small pieces of data stored in the user's browser by a website to remember information such as login sessions, preferences, or activity across multiple visits.

8) Write the difference between **Cookies**, **LocalStorage** **SessionStorage**

Feature	Cookies	LocalStorage	SessionStorage
Storage Size	Very small (~4 KB)	Large (5–10 MB)	Medium (5 MB)
Auto-sent to Server?	Yes (sent with every request)	No	No
Expires	Can set custom expiry or session-only	Never expires until cleared	Expires when tab/browser closes
Where Stored?	Browser + sent to server	Browser only	Browser only (per tab)
Access	Server + JS (unless HttpOnly)	Only JS	Only JS
Security Flags	Yes (HttpOnly, Secure, SameSite)	None	None
Best For	Login/session management, JWT (in HttpOnly cookie)	Storing long-term non-sensitive data	Temporary per-tab data (forms, steps)
Use in Authentication?	Recommended (most secure with HttpOnly)	Not recommended	Not recommended
Lifetime	Developer decides	Persistent	Tab/session only

(Session (This belongs to the **server**.) and Session Storage (This belongs to the **browser**.) both are different)

9) Cookies vs JWT vs Sessions

Feature	Cookies	Sessions
Stored in	Browser	Server
Sent to server automatically	Yes	Yes (via session ID)
Contains	Small data or session ID	Session ID (server holds user data)
Security	Supports HttpOnly, Secure, SameSite	Very secure (server-controlled)
Type	Storage mechanism	Server-side authentication method
Stateful/Stateless	Depends on use	Stateful